

فاز ۶: شبکه‌های کامپیوتری (Computer)

(Networks) – ویژه بک‌اند کاران

این بخش برای به توسعه‌دهنده بک‌اند حیاتی، مخصوصاً تو دنیای اینترپرایز که با REST API، میکروسرویس‌ها، پروتکل‌های امن و Load Balancer سروکار داری. مصاحبه‌کننده ازت انتظار داره بدونی وقتی به ریکوئست از فرانت‌اند میزنه بیرون، چه مسیری رو طی می‌کنه تا به بک‌اند تو برسه.

6.1 مدل OSI و TCP/IP

سوال: مدل OSI چیه و چرا باید بدونیمش؟

جواب: مدل OSI به مدل مرجع ۷ لایه‌س که ارتباطات شبکه رو استاندارد می‌کنه. هر لایه وظیفه مشخصی داره و با لایه بالا و پایین خودش در تعامله.

#	لایه (Layer)	وظیفه اصلی	مثال‌ها / پروتکل‌ها
۷	Application	رابط کاربر و شبکه	HTTP, HTTPS, FTP, SMTP, DNS
۶	Presentation	ترجمه، رمزنگاری، فشرده‌سازی	SSL/TLS, ASCII, JPEG
۵	Session	مدیریت نشست‌ها (Session)	NetBIOS, RPC
۴	Transport	انتقال مطمئن داده (Port)	TCP, UDP
۳	Network	مسیریابی بسته‌ها (IP)	IP, ICMP, Router
۲	Data Link	آدرس‌دهی فیزیکی (MAC)	Ethernet, Switch
۱	Physical	انتقال سیگنال‌های الکتریکی/نوری	کابل‌ها، Hub

اما مدل ساده‌تر TCP/IP که عملاً استفاده میشه ۴ لایه‌س:

•Application: ترکیب لایه‌های ۵-۷ (HTTP و غیره)

•Transport: لایه ۴ (TCP/UDP)

Internet: لایه ۳ (IP)

Network Access: ترکیب لایه‌های ۱-۲

? سوال مصاحبه‌ای ۱: توی مرورگر `https://example.com` رو میزنی. از لحظه Enter تا نمایش صفحه، چه اتفاقی میفته؟ (سوال طلایی و حیاتی)

جواب کامل:

1. DNS Lookup: مرورگر اول کش خودش، بعد کش OS و در نهایت DNS Resolver رو چک می‌کنه تا اسم دامنه (`example.com`) رو به IP تبدیل کنه.

2. TCP Connection (SYN, SYN-ACK, ACK): مرورگر یه درخواست اتصال TCP روی پورت ۴۴۳ (HTTPS) به اون IP می‌فرسته. این فرایند به "دست‌دهی سه‌مرحله‌ای" معروفه.

3. TLS Handshake (برای HTTPS): یه فرایند جداگونه برای تبادل گواهی (Certificate) و توافق روی کلیدهای رمزنگاری انجام میشه.

4. ارسال درخواست HTTP: مرورگر یه ریکوئست HTTP (مثلاً `GET /index.html`) می‌سازه و از طریق این تونل امن می‌فرسته.

5. پاسخ سرور: وب‌سرور (مثل Nginx) ریکوئست رو می‌گیره، به بک‌اند (مثل Spring Boot) پاس میده، Response می‌سازه و برمی‌گردونه.

6. رندر صفحه: مرورگر HTML رو می‌گیره، CSS و JS‌های ضمیمه رو هم دانلود می‌کنه، و صفحه رو رندر می‌کنه.

6.2 پروتکل‌های TCP و UDP (قلب لایه Transport)

? سوال مصاحبه‌ای ۲: تفاوت TCP و UDP چیه؟ کی کدوم رو استفاده می‌کنیم؟

جواب:

(TCP (Transmission Control Protocol: اتصال‌گرا و مطمئن.

• تحویل بسته‌ها رو تضمین می‌کنه (با Acknowledgement).

• ترتیب بسته‌ها رو حفظ می‌کنه.

• کنترل جریان (Flow Control) و ازدحام (Congestion Control) داره.

• سربار (Overhead) بیشتری داره و کندتره.

• مناسب برای: HTTP، ایمیل، انتقال فایل (FTP)، هرچیزی که گم شدن یه بیتش فاجعه‌ست.

(UDP (User Datagram Protocol: غیراتصال‌گرا و نامطمئن.

• بسته رو می‌فرسته، بدون تضمین برای رسیدن یا ترتیب.

• خیلی سریعتره و سربارش کمه.

• مناسب برای: استریم ویدئو، بازی‌های آنلاین، DNS، VoIP. (از دست رفتن چند بسته اهمیتی نداره، سرعت مهمه).

? سوال مصاحبه‌ای ۳: Handshake سه مرحله‌ای TCP (Three-way Handshake) رو

توضیح بده

جواب: فرآیندی که قبل از ارسال داده، یک اتصال TCP برقرار می‌کنه:

1.SYN: کلاینت به بسته با پرچم SYN (همگام‌سازی) به سرور می‌فرسته. ("میخوام وصل شم، شماره ترتیب من X هست.")

2.SYN-ACK: سرور با به بسته با پرچم SYN و ACK (تأیید) جواب می‌دهد. ("باشه. شماره ترتیب من Y هست، و منتظر X+1 هستم.")

3.ACK: کلاینت به بسته با پرچم ACK می‌فرسته. ("رسید. شروع کنیم.")

6.3 پروتکل HTTP (زبان مادری بک‌اند)

ساختار یک درخواست HTTP:

```
1 GET /api/users/1 HTTP/1.1
2 Host: www.example.com
3 Authorization: Bearer <token>
4 Content-Type: application/json
5 Accept: application/json
6
7 (empty line - body for POST/PUT)
```

? سوال مصاحبه‌ای ۴: متدهای اصلی (Verbs) HTTP رو با کاربردها توضیح بده

• GET: دریافت یک منبع (Resource). (امن و Idempotent)

• POST: ایجاد یک منبع جدید. (نه امن، نه Idempotent)

• PUT: جایگزینی کامل یک منبع موجود. (Idempotent)

• PATCH: به‌روزرسانی بخشی از یک منبع. (نه Idempotent لزوماً)

• DELETE: حذف یک منبع. (Idempotent)

تفاوت Safe و Idempotent:

- Safe: عملیات فقط خواندنی است و سرور رو تغییر نمیده (GET, HEAD, OPTIONS).
- Idempotent (خودتوان): یعنی اگر یه ریکوئست رو چند بار بفرستی، نتیجه فرقی نکنه. (GET, PUT, DELETE).
- POST خودتوان نیست (چندتا POST = چندتا رکورد جدید).

? سوال مصاحبه‌ای ۵: Status Code های مهم و پرتکرار رو نام ببر

:2xx (Success)

- OK 200: موفقیت در GET/PUT.
- Created 201: موفقیت در ساخت منبع جدید (POST).
- No Content 204: عملیات موفق ولی بدنه پاسخ خالیه (مثلاً بعد از DELETE).

:3xx (Redirection)

- Moved Permanently 301: تغییر مسیر دائم.

- Found 302: تغییر مسیر موقت.

:4xx (Client Error)

- Bad Request 400: درخواست مشکل داره (مثلاً JSON اشتباه).
- Unauthorized 401: احراز هویت نشده (لاگین نکرده).
- Forbidden 403: دسترسی نداره (لاگین کرده ولی نقشش اجازه نداره).
- Not Found 404: منبع پیدا نشد.
- Method Not Allowed 405: متد اشتباهی (مثلاً GET به جای POST).

:5xx (Server Error)

• **500 Internal Server Error:** خطای عمومی و پیش‌بینی نشده سرور (یه Exception خورده!).

• **502 Bad Gateway:** سرور بالادستی پاسخ نامعتبر داده.

• **503 Service Unavailable:** سرور موقتاً در دسترس نیست.

6.4 مفاهیم پیشرفته و پرتکرار شبکه

? سوال مصاحبه‌ای ۶: REST API چیه؟ چه ویژگی‌هایی داره؟

جواب: (REST (Representational State Transfer) به سبک معماری برای طراحی API‌های تحت وب هست.

• **Stateless** (بی‌وضعیت): هر درخواست از کلاینت به سرور باید تمام اطلاعات لازم رو داشته باشه. سرور وضعیت

(Session) کلاینت رو ذخیره نمی‌کنه. (این فلسفه JWT هم هست).

• **Resource-Based:** همه چیز یک منبع (Resource) محسوب میشه و با URL مشخص میشه (users/1/).

• استفاده از HTTP Verbs به صورت معنی‌دار: (GET برای خواندن، POST برای ساختن...).

• نمایش‌های چندگانه: به منبع می‌تونه با فرمت‌های مختلف (JSON, XML) برگردونده بشه.

? سوال مصاحبه‌ای ۷: تفاوت بین HTTP/1.1 و HTTP/2 رو توضیح بده

HTTP/1.1:

• **Head-of-Line Blocking:** هر اتصال TCP فقط یک درخواست رو در یک زمان پردازش می‌کنه.

• محدودیت اتصال: مرورگرها مجبورن ۶-۸ اتصال همزمان به هر دامنه باز کنن.

• متن‌گرا (Text-based).

:HTTP/2

• Multiplexing (تقسیم‌بندی): چندین درخواست و پاسخ می‌تونن همزمان روی یک اتصال TCP جریان داشته باشن. (مشکل Head-of-Line Blocking رو در لایه Application حل کرد).

• Header Compression (HPACK): هدرها فشرده میشن.

• Server Push: سرور می‌تونه منابعی که کلاینت نیاز داره رو قبل از درخواست ارسال کنه.

• باینری (Binary).

? سوال مصاحبه‌ای ۸: CORS (Cross-Origin Resource Sharing) چیه و چرا وجود داره؟

جواب: یه مکانیزم امنیتی در مرورگرهاست (Same-Origin Policy). مرورگرها به طور پیش‌فرض نمی‌ذارن یه اسکریپت از دامنه A، به API دامنه B درخواست بده.

مشکل ما: وقتی فرانت‌اند (مثلاً روی localhost:3000) و بک‌اند (روی localhost:8080) جدا هستن.

راه حل از سمت بک‌اند: سرور باید در هدر پاسخ، Access-Control-Allow-Origin رو با دامنه مجاز (یا * برای همه) بفرسته. برای متدهای خاص مثل PUT/DELETE، یه درخواست "پیش‌پرواز" (Preflight) با متد OPTIONS چک می‌کنه که اجازه‌ها رو بررسی کنه.

6.5 مفاهیم امنیتی شبکه (مکمل فاز امنیت)

• SSL/TLS: پروتکل رمزنگاری که امنیت رو در لایه Transport فراهم می‌کنه. اسم رمز HTTPS هست. کاراش: (۱) رمزنگاری داده، (۲) تضمین یکپارچگی داده، (۳) احراز هویت سرور (با Certificate).

• DNS (Domain Name System): دفترچه تلفن اینترنت. اسم دامنه را به IP تبدیل می‌کنه.